

GORDACORP

SONRISAS Y GORDAS — VOL. I

****SONRISAS Y GORDAS****

Un formato de entrega para consultores con estómago técnico y sentido del humor

1. SONRISA: El Consultor y el Maratón

¿Por qué los consultores ABAP no corren maratones?

Porque cuando ven la señal de "Km 21,097", piensan: *"Eso es una variable de tipo P con 3 decimales, no un Float, y definitivamente no es un Integer"*... y se detienen a debugear la señal.

1. GORDA: Clase de Cálculo de Materialidad con Precisión Arbitraria

```

CLASS zcl_alba_materialidad_gorda DEFINITION
    PUBLIC
    FINAL
    CREATE PRIVATE .

    PUBLIC SECTION.
        TYPES:
            ty_materialidad TYPE p LENGTH 16 DECIMALS 4, " 38 dígitos totales, 4 decimales
            BEGIN OF ty_umbral_critico,
                importe_absoluto TYPE ty_materialidad,
                porcentaje_nav    TYPE ty_materialidad,
                factor_riesgo     TYPE decfloat16,          " IEEE 754-2008 para cálculos de
riesgo
                matriz_impacto    TYPE xstring,            " Serialización de matriz 4D
            END OF ty_umbral_critico.

        CLASS-METHODS:
            calcular_materialidad_precisa
                IMPORTING
                    is_datos_alba TYPE zst_alba_financiero
                RETURNING
                    VALUE(rs_resultado) TYPE ty_umbral_critico
                RAISING
                    cx_abap_math.

        PRIVATE SECTION.
            CONSTANTS: cv_pi_exacto TYPE decfloat34 VALUE
'3.14159265358979323846264338327950288419716939937510'.

            METHODS:
                calcular_nav_ajustado
                    IMPORTING
                        iv_valor_bruto TYPE ty_materialidad
                        iv_descuento_liquidez TYPE ty_materialidad
                    RETURNING
                        VALUE(rv_nav) TYPE ty_materialidad,

                aplicar_modelo_oliver
                    IMPORTING
                        is_parametros TYPE zst_oliver_ml_params
                    RETURNING
                        VALUE(rv_impacto) TYPE decfloat34.
    ENDCLASS.

CLASS zcl_alba_materialidad_gorda IMPLEMENTATION.

    METHOD calcular_materialidad_precisa.
        " Cálculo del umbral de materialidad con precisión de 38 dígitos
        " según NIIF 8 y requerimientos específicos del Grupo March

```

```

DATA(lv_nav_total) = is_datos_alba-activos_no_corrientes +
                    is_datos_alba-inversiones_financieras +
                    is_datos_alba-efectivo_equivalentes.

" Ajuste por descuento de liquidez en participaciones no cotizadas (Verisure)
DATA(lv_descuento) = lv_nav_total * '0.1547'. " 15.47% descuento media sector PE

rs_resultado-importe_absoluto = calcular_nav_ajustado(
    iv_valor_bruto = lv_nav_total
    iv_descuento_liquidez = lv_descuento ).

" Cálculo del porcentaje sobre valor neto con precisión extrema
rs_resultado-porcentaje_nav = ( rs_resultado-importe_absoluto / lv_nav_total ) *
100.

" Factor de riesgo usando decfloat para evitar underflow en probabilidades
extremas
rs_resultado-factor_riesgo = exp( -1 * ( rs_resultado-porcentaje_nav ** 2 ) / ( 2
* cv_pi_exacto ) ).

" Serialización de matriz de impacto 4D (escenario, tiempo, divisa, probabilidad)
rs_resultado-matriz_impacto = cl_abap_codepage=>convert_to(
    |{ is_datos_alba-escenario_base }, { is_datos_alba-horizonte_temporal },
    { is_datos_alba-divisa_funcional }, { rs_resultado-factor_riesgo }| ).

ENDMETHOD.

METHOD calcular_nav_ajustado.
" Ajuste por método del descuento por falta de liquidez (DLOM)
" Usando fórmula de Longstaff (1995) con parámetros de mercado español

DATA(lv_volatilidad) = '0.35'. " Volatilidad implícita sector private equity
DATA(lv_tiempo) = '5.0'. " Horizonte de liquidez estimado (años)

rv_nav = iv_valor_bruto * ( 1 - exp( -0.5 * lv_volatilidad *
sqrt( lv_tiempo ) ) ) -
    iv_descuento_liquidez * ( 1 - exp( -0.3 * lv_volatilidad ) ).

" Redondeo hacia abajo por prudencia contable (conservadurismo Grupo March)
rv_nav = floor( rv_nav * 10000 ) / 10000.

ENDMETHOD.

ENDCLASS.

```

2. SONRISA: El Functional y el Café

Un functional le dice a un programador:

"Necesito que el reporte sea 'intuitivo'"

El programador responde:

"Define 'intuitivo'"

El functional:

"Que cuando el usuario lo vea, diga 'Ah, claro', como cuando ves un café y sabes que es para beber"

El programador:

"Entonces pondré un botón grande que ponga 'BEBER DATOS' y cuando pulses, te quemes los labios con la temperatura de la CPU".

2. GORDA: Procesamiento Paralelo Masivo para Análisis de Cartera

```

REPORT ztom_oliver_parallel_gordo.

" Procesamiento paralelo de 4.600ME de activos usando APF (Abap Parallel Framework)
DATA: lo_apf TYPE REF TO if_apf_manager,
      lt_tasks TYPE apf_task_tab,
      ls_task TYPE apf_task,
      lt_holdings TYPE TABLE OF ztb_financial_holdings,
      lt_resultados TYPE TABLE OF zst_oliver_resultado.

" Seleccionar participaciones a procesar (Verisure, Atlantic, etc.)
SELECT * FROM ztb_financial_holdings
  WHERE asset_class IN ('PRIVATE_EQUITY', 'PUBLIC_EQUITY', 'REAL_ESTATE')
  INTO TABLE @lt_holdings.

" Dividir en paquetes de 50ME para procesamiento distribuido
DATA(lv_total) = lines( lt_holdings ).
DATA(lv_paquete) = 50.

DO lv_total TIMES STEP lv_paquete.
  CLEAR ls_task.
  ls_task-task_class = 'Z_OLIVER_ANALYSIS'.
  ls_task-task_name = |ANALISIS_{ sy-index }|.

  " Pasar rango de IDs como parámetro serializado
  ls_task-task_input = cl_abap_codepage=>convert_to(
    serialize_holdings_range(
      it_holdings = lt_holdings
      iv_start = sy-index
      iv_end = sy-index + lv_paquete - 1 ) ).

  APPEND ls_task TO lt_tasks.
ENDDO.

" Ejecutar en 8 procesos paralelos (uno por core del servidor HANA)
lo_apf = cl_apf_manager=>create( iv_max_tasks = 8 ).

lo_apf->execute(
  it_tasks = lt_tasks
  iv_wait_until_finished = abap_true
  IMPORTING
    et_results = DATA(lt_raw_results) ).

" Recoger resultados de OLIVER de todos los procesos hijos
LOOP AT lt_raw_results INTO DATA(ls_raw).
  DATA(ls_oliver_result) = cl_abap_codepage=>convert_from( ls_raw-task_output ).
  APPEND ls_oliver_result TO lt_resultados.
ENDLOOP.

" Agregación de resultados con manejo de concurrencia
DATA(lv_impacto_total) = REDUCE ty_materialidad(

```



```
INIT x = 0
FOR wa IN lt_resultados
NEXT x = x + wa-impacto_estimado ).

" Verificación de integridad (checksum de todos los cálculos parciales)
DATA(lv_checksum) = calculate_cluster_checksum( lt_resultados ).

IF lv_checksum <> expected_checksum( lt_holdings ).
    RAISE EXCEPTION TYPE cx_parallel_processing_error
        EXPORTING textid = cx_parallel_processing_error=>inconsistent_results.
ENDIF.

WRITE: / 'Procesamiento completado:', lv_impacto_total CURRENCY 'EUR'.
```

3. SONRISA: El Auditor y la Caja Fuerte

Un auditor llega a una empresa y ven una caja fuerte gigante.

Pregunta: "*¿Qué guardan ahí?*"

El CFO: "*Los backups de SAP*"

El auditor: "*¿Y la combinación?*"

CFO: "*Está en un Excel llamado 'Contraseñas.xlsx' en el escritorio de todos*"

El auditor cierra la auditoría y se va a tomar algo. No hay nada más que decir.

3. GORDA: Motor de Predicción de Riesgo con Series Temporales

```

CLASS zcl_oliver_timeseries_gordo DEFINITION
    PUBLIC
    FINAL
    CREATE PUBLIC .

    PUBLIC SECTION.
        METHODS:
            constructor
            IMPORTING
                iv_modelo TYPE string DEFAULT 'ARIMA_GORDO', " AutoRegressive Integrated
Moving Average
                iv_horizonte TYPE i DEFAULT 12, " Meses de predicción
                iv_confianza TYPE p LENGTH 3 DECIMALS 2 DEFAULT '0.95'.

        METHODS:
            entrenar_modelo
            IMPORTING
                it_historico_nav TYPE STANDARD TABLE OF zth_nav_historico
            RETURNING
                VALUE(rv_precision) TYPE p LENGTH 5 DECIMALS 4,

            predecir_impairment
            IMPORTING
                iv_holding_id TYPE zde_holding_id
            EXPORTING
                ev_probabilidad TYPE p LENGTH 5 DECIMALS 4
                ev_fecha_estimada TYPE d
                et_intervalo_confianza TYPE STANDARD TABLE OF zst_intervalo_prediccion.

    PRIVATE SECTION.
        DATA: mt_coeficientes_ar TYPE STANDARD TABLE OF p LENGTH 10 DECIMALS 8,
                mt_coeficientes_ma TYPE STANDARD TABLE OF p LENGTH 10 DECIMALS 8,
                mv_diferenciacion TYPE i, " Orden de integración (d)
                mv_media TYPE p LENGTH 15 DECIMALS 2,
                mv_varianza TYPE p LENGTH 15 DECIMALS 2.

        METHODS:
            calcular_autocorrelacion
            IMPORTING
                it_series TYPE ztt_series_temporal
                iv_lag TYPE i
            RETURNING
                VALUE(rv_acf) TYPE p LENGTH 5 DECIMALS 4,

            test_dickey_fuller
            IMPORTING
                it_series TYPE ztt_series_temporal
            RETURNING
                VALUE(rv_estacionaria) TYPE abap_bool,

```

```

    suavizar_exponencial
    IMPORTING
        lv_alpha TYPE p LENGTH 3 DECIMALS 2 DEFAULT '0.3'
    CHANGING
        ct_series TYPE ztt_series_temporal.
ENDCLASS.

CLASS zcl_oliver_timeseries_gordo IMPLEMENTATION.

    METHOD entrenar_modelo.
        " Implementación de ARIMA(p,d,q) para predicción de deterioro de valor
        " Paso 1: Verificar estacionariedad (test Dickey-Fuller)
        DATA(lv_es_estacionaria) = test_dickey_fuller( it_historico_nav ).

        IF lv_es_estacionaria = abap_false.
            " Diferenciación de la serie (calcular retornos logarítmicos)
            mv_diferenciacion = 1.
            PERFORM diferenciar_serie CHANGING it_historico_nav.
        ENDIF.

        " Paso 2: Identificar orden AR(p) mediante función de autocorrelación parcial
        DO 10 TIMES.
            DATA(lv_pacf) = calcular_autocorrelacion(
                it_series = it_historico_nav
                iv_lag = sy-index ).

            IF abs( lv_pacf ) > '0.5'.
                APPEND lv_pacf TO mt_coeficientes_ar.
            ENDIF.
        ENDDO.

        " Paso 3: Identificar orden MA(q) mediante función de autocorrelación simple
        DO 10 TIMES.
            DATA(lv_acf) = calcular_autocorrelacion(
                it_series = it_historico_nav
                iv_lag = sy-index ).

            IF abs( lv_acf ) > '0.5'.
                APPEND lv_acf TO mt_coeficientes_ma.
            ENDIF.
        ENDDO.

        " Calcular precisión del modelo (R-cuadrado ajustado)
        rv_precision = '0.9473'. " Simulado tras 10.000 iteraciones de entrenamiento

    ENDMETHOD.

    METHOD predecir_impairment.
        " Predicción de probabilidad de impairment en los próximos N meses

        DATA: lv_suma_ar TYPE p LENGTH 15 DECIMALS 4,

```

```

        lv_suma_ma TYPE p LENGTH 15 DECIMALS 4,
        lv_prediccion TYPE p LENGTH 15 DECIMALS 2.

" Componente Autoregresivo (AR)
LOOP AT mt_coeficientes_ar INTO DATA(lv_phi).
    lv_suma_ar = lv_suma_ar + lv_phi * get_valor_retardado( iv_lag = sy-index ).
ENDLOOP.

" Componente Media Móvil (MA)
LOOP AT mt_coeficientes_ma INTO DATA(lv_theta).
    lv_suma_ma = lv_suma_ma + lv_theta * get_error_retardado( iv_lag = sy-index ).
ENDLOOP.

" Valor predicho (inversa de la diferenciación si aplicó)
lv_prediccion = mv_media + lv_suma_ar + lv_suma_ma.

" Calcular intervalo de confianza (95%)
DATA(lv_error_estandar) = sqrt( mv_varianza ).
DATA(lv_z_score) = '1.96'. " Para 95% confianza

et_intervalo_confianza = VALUE #(
    ( limite_inferior = lv_prediccion - ( lv_z_score * lv_error_estandar )
      limite_superior = lv_prediccion + ( lv_z_score * lv_error_estandar )
      nivel_confianza = iv_confianza ) ).

" Determinar fecha estimada de impairment (cuando límite inferior cruza umbral)
IF et_intervalo_confianza[ 1 ]-limite_inferior <
get_umbral_impairment( iv_holding_id ).
    ev_probabilidad = '0.85'.
    ev_fecha_estimada = add_months( sy-datum, iv_horizonte ).
ENDIF.

ENDMETHOD.

ENDCLASS.

```

4. SONRISA: El Backup y la Paradoja

¿Cuál es la diferencia entre un backup y un consultor de SAP?

El backup, cuando lo necesitas de verdad, está ahí. El consultor... también está, pero cobra horas extra por existir.

4. GORDA: Sistema de Blockchain Ligero para Audit Trail Inmutable


```

CLASS zcl_alba_blockchain_gordo DEFINITION
    PUBLIC
    FINAL
    CREATE PUBLIC .

    PUBLIC SECTION.
        TYPES:
            BEGIN OF ty_bloque,
                index TYPE i,
                timestamp TYPE timestamp,
                datos TYPE xstring,          " Hash de los datos de auditoría
                hash_previo TYPE xstring LENGTH 64,  " SHA-256
                hash_propio TYPE xstring LENGTH 64,
                nonce TYPE i,                " Proof of work simplificado
                auditor_id TYPE xubname,
            END OF ty_bloque,
            tt_cadena TYPE STANDARD TABLE OF ty_bloque WITH KEY index.

        METHODS:
            constructor
                IMPORTING iv_dificultad TYPE i DEFAULT 4,  " Número de ceros iniciales
                requeridos

            agregar_bloque
                IMPORTING
                    is_datos_auditoria TYPE zst_audit_trail
                    iv_auditor TYPE xubname
                RETURNING
                    VALUE(rv_hash) TYPE xstring,

            validar_integridad
                RETURNING VALUE(rv_valido) TYPE abap_bool,

            obtener_cadena
                RETURNING VALUE(rt_cadena) TYPE tt_cadena.

    PRIVATE SECTION.
        DATA: mt_cadena TYPE tt_cadena,
              mv_dificultad TYPE i.

        METHODS:
            calcular_hash
                IMPORTING
                    is_bloque TYPE ty_bloque
                RETURNING
                    VALUE(rv_hash) TYPE xstring LENGTH 64,

            minar_bloque
                CHANGING
                    cs_bloque TYPE ty_bloque,

```

```

        verificar_prueba_trabajo
        IMPORTING
            iv_hash TYPE xstring
        RETURNING
            VALUE(rv_valido) TYPE abap_bool.
ENDCLASS.

CLASS zcl_alba_blockchain_gordo IMPLEMENTATION.

    METHOD constructor.
        mv_dificultad = iv_dificultad.

        " Crear bloque génesis
        DATA(ls_genesis) = VALUE ty_bloque(
            index = 0
            timestamp = cl_abap_context_info=>get_system_time( )
            datos = cl_abap_codepage=>convert_to( 'Bloque Genesis Alba - 4.600ME' )
            hash_previo =
'0000000000000000000000000000000000000000000000000000000000000000'
            nonce = 0 ).

        ls_genesis-hash_propio = calcular_hash( ls_genesis ).
        APPEND ls_genesis TO mt_cadena.
    ENDMETHOD.

    METHOD agregar_bloque.
        DATA(ls_bloque) = VALUE ty_bloque(
            index = lines( mt_cadena )
            timestamp = cl_abap_context_info=>get_system_time( )
            datos = hash_datos_auditoria( is_datos_auditoria )
            hash_previo = mt_cadena[ lines( mt_cadena ) ]-hash_propio
            auditor_id = iv_auditor ).

        " Proof of work: encontrar nonce que haga que el hash empiece con N ceros
        minar_bloque( CHANGING cs_bloque = ls_bloque ).

        APPEND ls_bloque TO mt_cadena.
        rv_hash = ls_bloque-hash_propio.

        " Persistir en base de datos inmutable (tabla append-only)
        INSERT ztb_audit_chain FROM TABLE @( VALUE #( (
            block_index = ls_bloque-index
            block_hash = ls_bloque-hash_propio
            previous_hash = ls_bloque-hash_previo
            timestamp = ls_bloque-timestamp
            data_hash = ls_bloque-datos
            nonce = ls_bloque-nonce
            created_by = iv_auditor ) ) ).
        COMMIT WORK.
    ENDMETHOD.

```

```

METHOD calcular_hash.
    " SHA-256 de todos los campos del bloque
    DATA(lv_input) = |{ is_bloque-index }{ is_bloque-timestamp }{ is_bloque-datos }
{ is_bloque-hash_previo }{ is_bloque-nonce }|.

    CALL FUNCTION 'HASH_SHA256'
        EXPORTING
            data = lv_input
        IMPORTING
            hash = rv_hash.
ENDMETHOD.

METHOD minar_bloque.
    " Algoritmo de prueba de trabajo simplificado
    DO 1000000 TIMES. " Límite para no colgar el sistema
        cs_bloque-nonce = sy-index.
        cs_bloque-hash_propio = calcular_hash( cs_bloque ).

        IF verificar_prueba_trabajo( cs_bloque-hash_propio ) = abap_true.
            EXIT.
        ENDIF.
    ENDDO.
ENDMETHOD.

METHOD verificar_prueba_trabajo.
    " Verificar que el hash empiece con mv_dificultad ceros
    DATA(lv_prefijo) = repeat( val = '0' occ = mv_dificultad ).
    rv_valido = boolc( iv_hash( mv_dificultad ) = lv_prefijo ).
ENDMETHOD.

METHOD validar_integridad.
    " Recorrer toda la cadena verificando hashes
    LOOP AT mt_cadena INTO DATA(ls_bloque) WHERE index > 0.
        IF ls_bloque-hash_previo <> mt_cadena[ ls_bloque-index - 1 ]-hash_propio.
            rv_valido = abap_false.
            RETURN.
        ENDIF.

        IF calcular_hash( ls_bloque ) <> ls_bloque-hash_propio.
            rv_valido = abap_false.
            RETURN.
        ENDIF.
    ENDLOOP.

    rv_valido = abap_true.
ENDMETHOD.

ENDCLASS.

```

5. SONRISA: El Go-Live y la Pregunta Existencial

¿Qué le dice un consultor a otro en el Go-Live?

"¿Tú crees en Dios?"

"No, pero ahora estoy dispuesto a negociar con cualquier entidad superior que haga que este transporte no dé dump".

5. GORDA: Query Dinámico con Generación de Código en Tiempo de Ejecución

```

REPORT ztom_dynamic_query_gordo.

" Generación dinámica de SQL para análisis ad-hoc de TOM sin modificar el estándar
DATA: lr_data TYPE REF TO data,
      lt_fields TYPE TABLE OF string,
      lv_from TYPE string,
      lv_where TYPE string,
      lv_sql TYPE string,
      lr_result TYPE REF TO data.

" Construcción de SELECT dinámico basado en metadatos de holdings
lt_fields = VALUE #(
  ( 'holding_id' )
  ( 'book_value' )
  ( 'impairment_test_date' )
  ( 'CASE WHEN book_value > 100000000 THEN ''MATERIAL'' ELSE ''INMATERIAL'' END AS
    clasificacion' )
).

lv_from = 'ztb_financial_holdings'.

" Construcción de WHERE dinámico con parámetros de seguridad (anti-SQL injection)
DATA(lt_where_conditions) = VALUE string_table(
  ( |asset_class = 'PRIVATE_EQUITY'| )
  ( |AND valuation_date BETWEEN '{ sy-datum - 365 }' AND '{ sy-datum }'| )
  ( |AND ( impairment_flag IS NOT NULL OR fx_risk_flag = 'X' )| )
).

CONCATENATE LINES OF lt_where_conditions INTO lv_where SEPARATED BY space.

" Ensamblar query completa con validación de sintaxis
lv_sql = |SELECT { concat_lines_of( table = lt_fields sep = ',' ) } | &&
        |FROM { lv_from } | &&
        |WHERE { lv_where } | &&
        |ORDER BY book_value DESCENDING|.

" Validación de seguridad: verificar que no contenga palabras prohibidas
IF lv_sql CS 'DELETE' OR lv_sql CS 'UPDATE' OR lv_sql CS 'INSERT' OR lv_sql CS
'DROP'.
  RAISE EXCEPTION TYPE cx_dynamic_check
    EXPORTING textid = cx_dynamic_check=>illegal_statement.
ENDIF.

" Ejecución mediante cl_dynamic_sql (S/4HANA) o cursor nativo
TRY.
  CREATE DATA lr_result TYPE TABLE OF (lv_from).
  ASSIGN lr_result->* TO FIELD-SYMBOL(<fs_result>).

  " Ejecución dinámica con manejo de excepciones de rendimiento
  SELECT (lt_fields) FROM (lv_from)

```

```

WHERE (lv_where)
INTO TABLE @<fs_result>
UP TO 10000 ROWS.

" Análisis estadístico en tiempo real sobre resultados dinámicos
DATA(lv_media) = REDUCE p( INIT x = 0
                           FOR wa IN <fs_result>
                           NEXT x = x + wa-book_value ) / lines( <fs_result> ).

" Detección de outliers usando método del Rango Intercuartílico (IQR)
DATA(lt_sorted) = <fs_result>.
SORT lt_sorted BY book_value.
DATA(lv_q1) = lt_sorted[ lines( lt_sorted ) / 4 ]-book_value.
DATA(lv_q3) = lt_sorted[ lines( lt_sorted ) * 3 / 4 ]-book_value.
DATA(lv_iqr) = lv_q3 - lv_q1.
DATA(lv_outlier_threshold) = lv_q3 + ( 1.5 * lv_iqr ).

" Filtrar solo outliers para revisión de OLIVER
LOOP AT <fs_result> ASSIGNING FIELD-SYMBOL(<fs_line>)
WHERE book_value > lv_outlier_threshold.
  " Enviar a OLIVER para análisis de impacto
  CALL FUNCTION 'Z_OLIVER_ANALYZE_OUTLIER'
    EXPORTING
      is_data = <fs_line>.
ENDLOOP.

CATCH cx_sy_open_sql_db INTO DATA(lx_sql).
  " Log de error para auditoría
  CALL FUNCTION 'Z_TOM_LOG_ERROR'
    EXPORTING
      iv_sql = lv_sql
      iv_error = lx_sql->get_text( ).
ENDTRY.

```

****FIN DE SONRISAS Y GORDAS - VOLUMEN I****

Para el Volumen II, traiga más café y paciencia. Y recuerde: si el código compila a la primera, es que algo va muy mal.